



使用Files工具类读取文件内容

北京理工大学计算机学院
金旭亮

文件的写入-1

```
//使用write方法写入文件
static void writeToFile(){
    Path textFile = Paths.get( first: "study.txt");
    String line1 = "好好学习, ";
    String line2 = "天天向上! ";
    List<String> lines = Arrays.asList(line1, line2);
    try {
        Files.write(textFile, lines, StandardCharsets.UTF_8);
    } catch (IOException ex) {
        ex.printStackTrace();
    }
    System.out.println("文件写入完毕! ");
}
```

ReadWriteFileDemo.java



如果文件不存在，会创建相应文件。

文件的写入-2

```
//使用writeString方法写入文件
static void useWriteString() throws IOException {
    Path testFilePath = Path.of( first: "poem.txt");
    String poem = ""
        龟虽寿
        神龟虽寿，犹有竟时；
        腾蛇乘雾，终为土灰。
        老骥伏枥，志在千里；
        烈士暮年，壮心不已。
        盈缩之期，不但在天；
        养怡之福，可得永年。
        幸甚至哉，歌以咏志。
        "";
    Files.writeString(testFilePath, poem);
    System.out.println("文件写入完毕! ");
}
```

这种方式，一次性地将所有数据写入到文件中，适合于不大的字符串。

注意：如果文件已经存在，则默认情况下它会被覆盖掉。

自我探索



```
public static Path writeString(Path path, CharSequence csq, OpenOption... options)
    throws IOException
{
    return writeString(path, csq, UTF_8.INSTANCE, options);
}
```

E StandardOpenOption

- f READ
- f CREATE
- f SPARSE
- f SYNC
- f DELETE_ON_CLOSE
- f TRUNCATE_EXISTING
- f WRITE
- f APPEND
- f CREATE_NEW
- f DSYNC

writeString方法有一个可选的参数，
可用于指定文件处理模式，比如，当文件
已存在时，是追加而不是覆盖。

请编写代码测试这一特性。



以下这段代码非常低效，你有办法优化它吗？

```
List<String> strings = List.of("one", "two", "three");  
// NO!  
for(String string : strings) {  
    Files.writeString(Path.of(filePath), csq: string + System.lineSeparator(), StandardOpenOption.APPEND);  
}
```

提示：

在循环中拼接字符串，是一种比较低效的方式。

文件写入-3

使用带有缓冲的写入方式，能提升IO效率，但要注意一定要及时Flush和Close，否则，就有可能出现数据丢失的现象。

推荐如示例所示，使用try with resources特性保证底层数据流一定会被关闭。

```
//使用BufferedWriter写入文件
static void useBufferedWriter() throws IOException {
    try (var writer :BufferedWriter = Files.newBufferedWriter(
        Path.of( first: "poem2.txt")))) {
        writer.write( str: "《山居秋暝》");
        writer.newLine();
        writer.write( str: "唐·王维");
        writer.newLine();
        writer.write( str: "空山新雨后，天气晚来秋。");
        writer.newLine();
        writer.write( str: "明月松间照，清泉石上流。");
        writer.newLine();
        writer.write( str: "竹喧归浣女，莲动下渔舟。");
        writer.newLine();
        writer.write( str: "随意春芳歇，王孙自可留。");
        writer.newLine();
    }
    System.out.println("文件写入完毕！");
}
```

一次读取所有行

```
static void readAllLinesFromFile() throws IOException {  
    Path filePath = Paths.get("poem.txt");  
    Files.readAllLines(filePath)  
        .forEach(System.out::println);  
}
```



使用这种方式，注意文件不要太大，否则，会占用太多的内存。

存取文件前先检查

```
public class CheckBeforeAccess {  
    public static void main(String[] args) throws IOException {  
        Path fileToAccess = Path.of("poem.txt");  
        if (isFileAccessible(fileToAccess)) {  
            System.out.println(Files.readString(fileToAccess));  
        }  
    }  
    1 usage  
    private static boolean isFileAccessible(Path file) {  
        return Files.isRegularFile(file) && Files.isReadable(file);  
    }  
}
```

Run: CheckBeforeAccess x

"C:\Program Files\Java\jdk-17"

《山居秋暝》

唐·王维

空山新雨后，天气晚来秋。

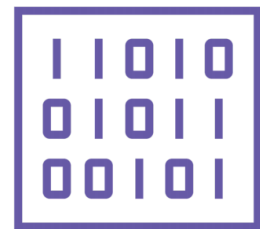
明月松间照，清泉石上流。

竹喧归浣女，莲动下渔舟。

随意春芳歇，王孙自可留。

使用这种方式，可以避免由于存取权限等问题，导致文件读不出来而抛出异常。

按二进制方式读取所有内容



`//使用readAllBytes()读取所有文件内容`

```
private static void readAllBytesDemo() throws IOException {  
    Path path = Paths.get( first: "poem.txt");  
    byte[] fileContentData = Files.readAllBytes(path);  
    System.out.println("读入字节数: " + fileContentData.length);  
    String fileContent = new String(fileContentData,  
        offset: 0, fileContentData.length, StandardCharsets.UTF_8);  
    System.out.println(fileContent);  
}
```

逐行读取-1

Files提供了newXXX()系列方法，返回一个使用内部数据缓存的文件存取对象，下面例子使用它循环读取每一行.....

```
private static void useBufferedReader() throws IOException {  
    Path source = Paths.get( first: "poem.txt");  
    //使用Files, 可以直接创建BufferedReader  
    try (BufferedReader reader = Files.newBufferedReader(source,  
        StandardCharsets.UTF_8)) {  
        String line = null;  
        while ((line = reader.readLine()) != null) {  
            System.out.println(line);  
        }  
    }  
}
```

逐行读取-2

使用`Files.lines()`方法获取一个字符串流 (`Stream<String>`) :

```
//使用lines方法逐行读取文本文件
private static void readFileLineByLine() throws IOException {
    Path filePath = Paths.get( first: ".", ...more: "poem.txt");
    try (Stream<String> lines = Files.lines(filePath)) {
        lines.forEach(System.out::println);
    }
}
```



使用这种方式，可以读取较大的文件，并且可以使用 Stream API，一边读入一边立即处理，推荐使用。